

AUGER: Automatically Generating Review Comments with Pre-training Models

Lingwei Li

Institute of Software, CAS, Univ. of
Chinese Academy of Sciences
Beijing, China
lilingwei20@mails.ucas.ac.cn

Li Yang*

Institute of Software, CAS
Beijing, China
yangli2017@iscas.ac.cn

Huaxi Jiang

Institute of Software, CAS, Univ. of
Chinese Academy of Sciences
Beijing, China
jianghuaxi19@mails.ucas.ac.cn

Jun Yan

State Key Laboratory of Computer
Science, Institute of Software, CAS,
Univ. of Chinese Academy of Sciences
Beijing, China
yanjun@ios.ac.cn

Tiejian Luo

Univ. of Chinese Academy of Sciences
Beijing, China
tjluo@ucas.ac.cn

Zihan Hua

Wuhan University, Univ. of Chinese
Academy of Sciences
Wuhan, China
2018302110434@whu.edu.cn

Geng Liang

Institute of Software, CAS
Beijing, China
lianggeng@iscas.ac.cn

Chun Zuo

Sinsoft Company Limited
Beijing, China
zuochun@sinsoft.com.cn

ABSTRACT

Code review is one of the best practices as a powerful safeguard for software quality. In practice, senior or highly skilled reviewers inspect source code and provide constructive comments, considering what authors may ignore, for example, some special cases. The collaborative validation between contributors results in code being highly qualified and less chance of bugs. However, since personal knowledge is limited and varies, the efficiency and effectiveness of code review practice are worthy of further improvement. In fact, it still takes a colossal and time-consuming effort to deliver useful review comments.

This paper explores a synergy of multiple practical review comments to enhance code review and proposes **AUGER (Automatically Generating Review comments)**: a review comments generator with pre-training models. We first collect empirical review data from 11 notable Java projects and construct a dataset of 10,882 code changes. By leveraging Text-to-Text Transfer Transformer (T5) models, the framework synthesizes valuable knowledge in the training stage and effectively outperforms baselines by **37.38%** in ROUGE-L, **29%** of our automatic review comments are considered useful according to prior studies. The inference generates just in 20 seconds and is also open to training further. Moreover, the performance also gets improved when thoroughly analyzed in case study.

*Corresponding author

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ESEC/FSE '22, November 14–18, 2022, Singapore, Singapore
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9413-0/22/11.
<https://doi.org/10.1145/3540250.3549099>

CCS CONCEPTS

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Review Comments, Code Review, Text Generation, Machine Learning

ACM Reference Format:

Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. 2022. AUGER: Automatically Generating Review Comments with Pre-training Models. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*, November 14–18, 2022, Singapore, Singapore. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3540250.3549099>

1 INTRODUCTION

Modern code review is considered effective in reducing programming defects and improving the quality of software in early phase of development [2, 37, 52]. Therefore, it is widely adopted nowadays in both open source and industry workflows [3]. Reviewers need to thoroughly understand the source code during code review activities and leave review comments to interact with developers. Figure 1.a shows an example of the review process. (A) First, developers create a pull request to submit code changes. (B) Then, reviewers receive the request and inspect the involved programming files. (C) Once they fully judge and disagree with some lines, (D) reviewers will leave messages and set up a conversation right behind them as in-line review comments. In summary, the assignment of reviewers is to double-check code changes, highlight review lines (bold font) and deliver useful review comments (Figure 1.b).

The nature of this cross-validation brings high quality to software [13]. For reviewers, they provide professional suggestions

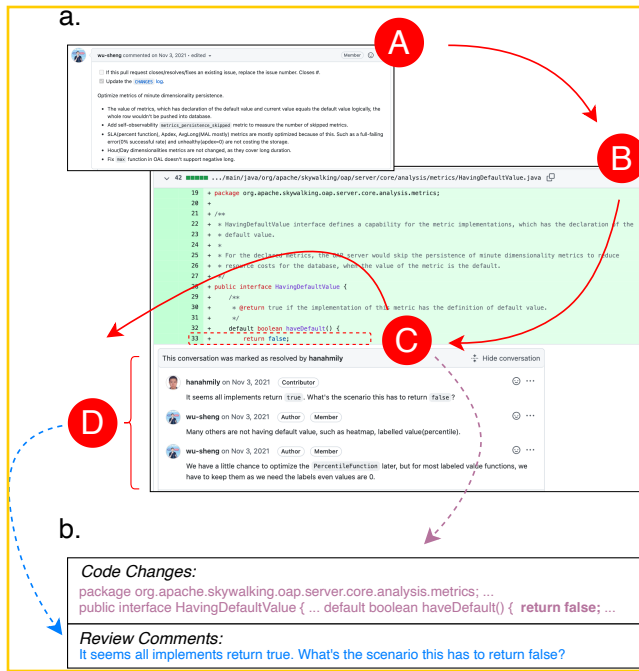


Figure 1: An Example of Code Review on GitHub

with their programming experience, which the author may not have been through [44]—for example, a convenient coding style, a particular test case, or a complicated application scenario. For authors, they can learn from perspectives other than themselves and have a higher chance to perform efficiently [4]. However, two factors limit the benefits:

1) The usefulness of existing review comments remains uncertain. Czerwonka et al. found that only 15% of review comments indicate a possible defect [14], and Bosu et al. reported that 34.50% of review comments from five major projects of Microsoft are non-useful [8]. Studies also proposed that low-quality review comments are considered useless and somehow mislead developers [3, 38]. Even so, it seems still inevitable in practice. On the one hand, both authors' and reviewers' programming knowledge is still limited since they can't think over all possible conditions. On the other hand, it is difficult to assign review requests to correct reviewers due to that limitation [58]. The assessment of useful review comments is controversial, too. Studies have argued that useful review comments can successfully identify functional issues or reflect other motivations, such as the social benefits of developers [2, 8]. However, research by Rahman et al. reported that review comments could be defined as useful only if they trigger code changes within ten lines after reviewing [44].

2) The heavy involvement of human effort is annoying and time-consuming. Researchers argued that code review might be the lengthiest part of the development [14, 37]. Reviewers have to change work context unwillingly and fully understand a source code from others before commenting. For authors, they can't move forward until reviewers send their review comments back [14]. Both of them are highly time-consuming in practice [3]. On average, it

costs developers 6 hours per week to handle code review issues [7]. Furthermore, due to the low efficiency, more contributors are assigned to the review process. Around 20% developers at Mozilla, a free software community, face heavy workloads, i.e., over ten patches/reviews per week beyond their own jobs [31]. In Microsoft Bing, industrial projects can undergo approximately 3,000 code reviews per month [45]. In addition, developers have to spend an extra effort to work as reviewers every time they participate in an unfamiliar project.

Prior studies have made attempts to solve these two issues of review comments. Balachandran proposed *Review-Bot*, which integrates the output of multiple static analysis tools to publish review comments automatically [3]. Rahman et al. reported that their *RevHelper* could directly predict whether review comments are useful or not [44]. Though competitive, they work as an indirect integration and validation, not for the review itself. Tufano et al.'s latest study focused on automated code review with automatic code revision and implemented comments generation as a sub-task, too [57]. However, their method remains coarse-grained and only aimed at finding defects in code functions, not review lines.

In this paper, we explore an unlimited and efficient code reviewer and propose a novel approach, called **AUGER**, to generate review comments automatically with continuous training at the first step. We leverage pre-training models to synthesize effective manual review comments, thanks to its proven efficiency for big data in Natural Language Processing. From 11 influential open-source Java projects in [GitHub](https://github.com/)¹ (the largest source code host [67]), we first collect those review pairs of code changes and review comments in 20K pull requests, which causes code revision. And then, we process data with heuristic methods and semantic augmentation in Data Preparation. With around 62K pieces of processed data, AUGER addresses the problem into three sub-processes: 1) Review Lines Tagging finds and highlights review lines that get revised after reviewing in code blocks with a unique leading review tag; 2) Cross Pre-training learns the inner correlations between code changes and review comments language with a masked language model from Text-To-Text Transfer Transformer (T5); 3) Comments Generation fine-tunes the pre-trained model further with selected pairs and finally transfers code changes in the programming language to review comments in natural language. Our experiment results show that AUGER achieves 22.97% in ROUGE-L and 4.32% in Perfect Prediction, which outperforms baseline by 37.38% and even 14 times. For efficiency, AUGER generates one piece of review comments as fast as 20 seconds. We also implement subsequent training and illustrate that AUGER can retrain freely in ablation. Moreover, we assess the usefulness of generated review comments with heuristic metrics from prior studies and present case study. We believe that AUGER has achieved our primary goal and is open to improvement in the future. The significant contributions of our paper are:

- We first formulate an issue of code review as a problem that generating review comments on manual-labelled problematic lines in code functions. To the best of our knowledge, this is the first study exploring an interactive automation by sharpening code changes.

¹<https://github.com/>

- To solve the problem, we propose AUGER, an synthetic and efficient generator to generate review comments automatically with instructive review tags, limitless training and immediate generation.
- We evaluate AUGER on 1,088 test data from code review practice in GitHub, and it performs effectively in both automatic evaluation and heuristic assessment.

The rest of this paper is introduced as follows. Section 2 describes the background and definition of the problem. Section 3 illustrates the detailed approach. Section 4 presents the experiments. Section 5 shows the evaluation of AUGER. Section 6 introduces the related work. Section 7 illustrates threats to validity. Section 8 makes a conclusion.

The AUGER² model and all experimental materials are publicly available to reproduce our results.

2 BACKGROUND

This section introduces the necessary background and the problem definition.

2.1 Review Documents

Modern code review is usually adopted in grand projects consisting of numerous documents [3, 37]. To simplify the illustration, we present involved review documents as several equations:

$$\begin{aligned} D &= \{d_1, d_2, \dots, d_n\}, \\ Sub_Code &= \{v_1^s, v_2^s, \dots, v_{ns}^s\} \in D, \\ Rev_Code &= \{v_1^r, v_2^r, \dots, v_{nr}^r\} \in D. \end{aligned} \quad (1)$$

The Equation 1 represents the main materials employed during the course. As mentioned above, code review will not commence until a submission of code changes. Hence, given a universal D of all documents d , we define these submitted code changes as Sub_Code . After reviewing, versions of the code that submitters may revise is defined as Rev_Code . v^s and v^r denotes code blocks divided by abstract syntax tree (AST) in these two texts, respectively.

2.2 Review Process

When review requests trigger, reviewers are assigned to inspect, analyze, and produce review comments in chronological order [45]. We describe the process with the following symbols:

$$\begin{aligned} P &= \{\vec{p}_1, \vec{p}_2, \dots, \vec{p}_m\}, \\ \vec{p}_i &= \langle v_j^s, T, C, \vec{I} \rangle \end{aligned} \quad (2)$$

The series of processes is defined as a set P in Equation 2. Each element $\vec{p}_i$ denotes one single reviewers' manipulation at that moment. For each $\vec{p}_i$, the inspector notices a code block v_j^s with possible issues in provided code changes, selects incorrect lines with a click, and leaves review comments C . To present it better, we abstract the click operation as a special leading tag T to distinguish those highlighted lines in the text. Besides, the system records relevant messages $\vec{I}$ such as timestamp and developer id for each submission.

2.3 Automatic Generation

The automatic generation task is to automate the review process based on materials defined in the above sections. Thus, we formulate our key idea with two equations:

$$\begin{aligned} \vec{R} &= \langle Sub_Code, P \rangle, \\ P_{auto} &:= \langle v_j^s, T \rangle \rightarrow C_{gen} \end{aligned} \quad (3)$$

In Equation 3, code review practice is first summarized as a combination of complete submitted code changes Sub_Code and reviewers' operation P from Equation 1 and Equation 2. The automatic process P_{auto} is defined as: a text generation function from code blocks v_j^s and review line tags T combination $\langle v_j^s, T \rangle$, mapping to an output of generated review comments C_{gen} , like the manual one C . In other words, the motivation of AUGER is to act as an effective automatic substitution P_{auto} for the human process P .

3 APPROACH

3.1 Overview

Figure 2 presents the framework of AUGER, which has five abstract components: A. Data Preparation (3.2); B. Review Lines Tagging (3.3); C. Cross Pre-training (3.4); D. Comments Generation (3.5); E. Application (3.6).

Given a universe of documents $D = \{d_1, d_2, \dots, d_n\}$ fetched from GitHub projects, Data Preparation selects and pre-processes the review code $Sub_Code = \{v_1^s, v_2^s, \dots, v_{ns}^s\}$ and review comments C . Review Lines Tagging highlights review lines with special tags T where revision will commence, according to Rev_Code . Cross Pre-training part learns the inner distribution of review text first, and then Comments Generation transfers the combination (code blocks Sub_Code with leading review tags T) into review comments C automatically. After fully pre-training and fine-tuning, AUGER P_{auto} can be applied to generate automatic review comments C_{gen} from new code changes Sub_Code and manual tags T in Application.

3.2 Data Preparation

1) Data Fetching: When fetching data from GitHub, we mainly follow the instruction of GitHub GraphQL API³. To ensure the effectiveness of review data with experienced review comments, we exactly fetch those review activities that indeed trigger code revisions [8]. Besides, we remove those reviews not for Java code. After eliminating invalid and repetitive data, triplets $\langle Sub_Code, Rev_Code, C \rangle$ are ready for further process. Figure 3 exhibits an example of Data Preparation: a pair of code changes and review comments input. For code text, we first remove source comments and then implement several processes such as splitting. Almost a same process is reimplemented for comments but an extra data augmentation afterwards.

2) Data Processing: We first check duplicates again to ensure that there is no in-train/in-test/cross-set duplicates causing data inflation [1]. Then we remove nearly 85%(67672) data consisting of two parts: 1) 40.77%(32348) reviews where code revision or review comments include content beyond functions. In terms of function-related review activities, they are out-of-range and thus considered noisy. 2) 44.52%(35324) " $\langle sub_code, comments \rangle$ " pairs where code

²<https://gitlab.com/ai-for-se-public-data/auger-fse-2022>

³<https://docs.github.com/en/graphql/>

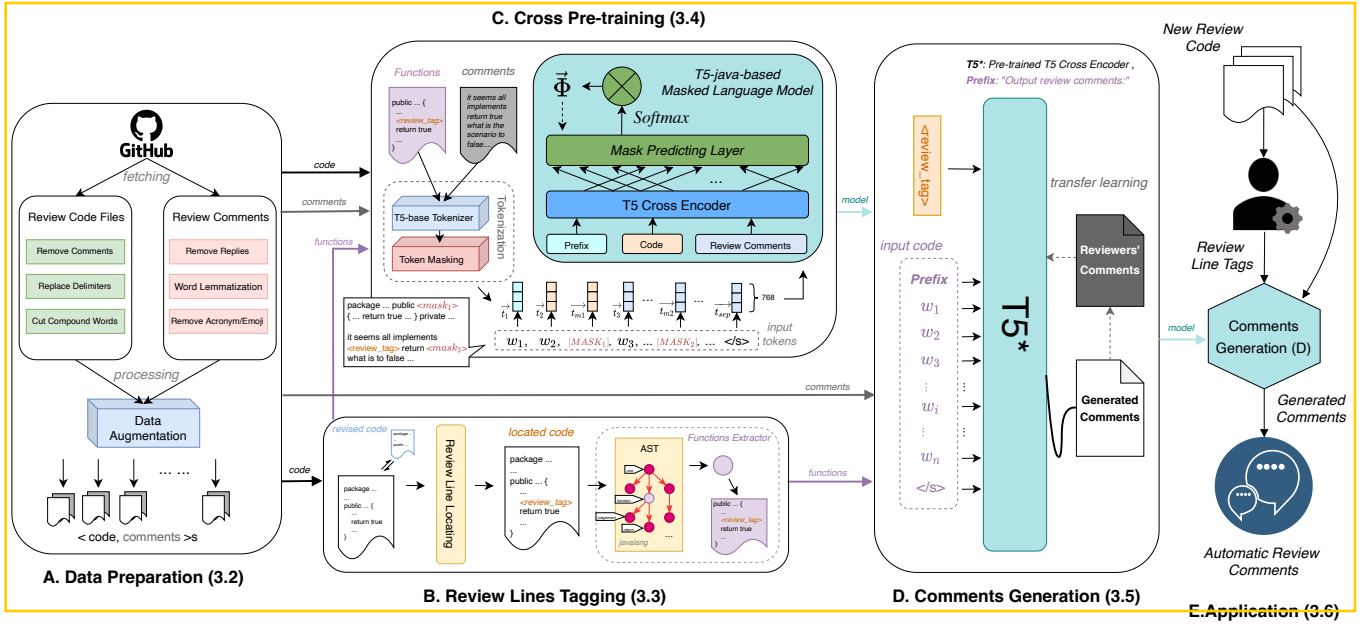


Figure 2: the Framework of AUGER

Code	<pre> /* Licensed to the Apache Software Foundation ... */ package org.apache.skywalking.oap.server.core.analysis.metrics; ... public interface HavingDefaultValue { /** @return true if the implementation has the definition of default value */ default boolean haveDefaultValue() { return false; } default boolean isDefaultValue() { return false; } } </pre>	1. remove comments: <pre> package org.apache.skywalking.oap.server.core.analysis.metrics; ... public interface HavingDefaultValue { public interface HavingDefaultValue { default boolean haveDefaultValue() { return false; } default boolean isDefaultValue() { return false; } } } </pre>	2. process (lowercase, split, lemmatize...): <pre> package org.apache.skywalking.oap.server.core.analysis.metrics; ... public interface have default value { default boolean have default () { return false ; default boolean is default value () { return false ; } } } </pre> 2. process (lowercase, acronym, emoji...): <pre> it seem all implement return true what be the scenario this have to return false </pre>
	Review 1: @XXX: It seems all implements return 'true'. What's the scenario this has to return 'false'? Review 2: Many others are not having default value, such as heatmap, labelled value(percentile).	1. remove replies: <pre> @XXX: It seems all implements return 'true'. What's the scenario this has to return 'false'? </pre>	3. data augmentation: <pre> it seem all implement return true what be the scenario this have to return false this seem all implement return true what be the scenario it have to return false it appear all implement return true what be the scenario this have to return false </pre>

Figure 3: An Example of Data Preparation

functions or review comments are shorter than three words or longer than 128. Some of them are considered noise, such as incomplete functions and useless replies like “fixed”. Besides, big functions are considered smelly [5] and too long for ML models, along with lengthy comments that include code blocks not in natural language. The rest 15%(11672) are regarded as raw data and ready for next process.

Then we convert all text into lowercase, split words and punctuation, and implement word lemmatization using NLTK⁴ toolkit (“seems” to “seem”, “is” to “be” in Figure 3). Then we process code and comments further so that models can quickly learn later.

For code text, we first totally remove source code comments with a heuristic filter. We preserve every punctuation except “.” owing to its representation of structure knowledge. In Java, functions and variables are usually named within long and compound words. In this case, we split them into fine-grained sub-words with the help

of WordNinja⁵. For example, from “HavingDefaultValue” to “having default value” in Figure 3.

For comments, we erase all signs such as “@” in natural language. Then acronyms are replaced by their full names referring to Oxford Dictionary⁶ (“What’s” to “What is” in Figure 3). After that, We remove all punctuation to preserve semantic words. The process of code fragments in comments is as same as in code text. Last, we select the first 3 sentences when review comments are too long.

3) Data Augmentation: This process is widely applied to handle the lack of training data in NLP [15, 28, 43]. Though we have fetched enough raw data, the amount decreases sharply after being filtered. Besides, studies reported that data augmentation is able to expand the influence of core knowledge in the training [15]. Hence, we boost triplets <Sub_Code, Rev_Code, C> by 9 times with only increasing review comments C using Wei et al. [62].

⁴<https://nltk.org/>⁵<https://github.com/keredson/wordninja/>⁶<https://public.oed.com/how-to-use-the-oed/abbreviations/>

3.3 Review Lines Tagging

Reviewers actually conduct reviewing on specific code lines. In practice, they highlight review lines first and then make review comments right after them. To be aligned with the fact, we synthesize this fine-grained review knowledge and reprocess the prepared code text to find out those review lines with a review tag T . Since we are only concerned about those code blocks indeed get revised afterward, this component is designed to remove irrelevant code blocks, too.

1) Review Lines Locating: In our implementation, we add a special token `<review_tag>` to the front of review lines. As shown in Equation 4, after receiving code text from Data Preparation, we first compare original code changes $Sub_Code = \{v_1^s, v_2^s, \dots, v_n^s\}$ and its revision $Rev_Code = \{v_1^r, v_2^r, \dots, v_n^r\}$ to find out different lines $Diff$. Then we only select valid modifications $Diff_{valid}$ with a series of line changes. Subsequently, we set `<review_tag>` at the beginning of it to mark where revision happened. This process follows the practice that developers leave review comments on chosen lines for collaborators easy to understand. Although knowing where to comment is also a problem, it takes as fast as one inference when generating several comments in a batch. Hence, people can select all code lines they want and choose which results to learn or improve. What's more, if review tags are predicted automatically, an additional error is brought to the overall performance.

$$\begin{aligned} Diff &= \{v^s | v^s \in Sub_Code\}, \\ Diff_{valid} &= \{v_i^s, v_{i+1}^s, \dots, v_{i+k}^s\}, \quad k \in N \end{aligned} \quad (4)$$

2) Functions Extractor: Studies have demonstrated that noise in input data can badly mislead the performance of language models [16, 30]. For safety, functions extractor is a tool to narrow the code range down to the least block v_i^s . First, we employ javalang⁷ to fomulate code files into abstract syntax trees (AST). In AST, the structure of code is arranged in a hierarchy instead of a inclusion. For example, a root node is at the top, while other functions, judgments, and statements follow it. Similar to prior works [36, 58], we extract code at the function height and select the one including target `<review_tag>`.

3.4 Cross Pre-training

Recent studies confirm that the masked language approach in pre-training can facilitate the performance of models in many NLP tasks [15, 61]. Therefore, we synthesize review knowledge by pre-training a better contextual representation and bridging the relationship between code and comments with cross pre-training technique.

1) Tokenization: We pair processed comments C (from Data Preparation) and functions v_i^s (from Review Lines Tagging) into $\langle v_i^s, C \rangle$. As pre-training usually comes into effect with numerous inputs, we also integrate raw data from Data Preparation to increase $\langle v_i^s, C \rangle$ directly. Then a T5-base tokenizer is employed to tokenize each of the text pairs (including `<review_tag>`) in one sentence. According to what BERT has proven effective in token masking [15], we randomly select 10% words to mask.

2) Mask Language Model: In this part, we build a T5-java-based Masked Language Model to do language masking for pre-training.

⁷<https://github.com/c2nes/javalang/>

So far, there has been a lot of varieties of T5 models [34]. We choose a version of CodeTrans [17] already trained on Java documents (from Hugging Face⁸). First, it works as a cross encoder by encoding both code and comment tokens in one sequence. For `<review_tag>`, we increase the embedding size by 1 to present it as a special token. Following the generation of T5, we write a prefix here as: "Generating review comments:" for all input data. Then we set a mask predicting layer to capture hidden states and output prediction vectors. Then, the softmax layer smooths it into probabilities $\tilde{\Phi}$.

$$\begin{aligned} \tilde{\Phi} &= \langle \phi_1, \phi_2, \dots, \phi_{|V|} \rangle, \quad 0 \leq \phi_1, \phi_2, \dots, \phi_{|V|} \leq 1 \\ \phi_1 + \phi_2 + \dots + \phi_{|V|} &= 1 \end{aligned} \quad (5)$$

$\phi_1, \phi_2, \dots, \phi_{|V|}$ respectively describes the possibility of the mask to be the first, the second, ..., the last word in the $|V|$ length vocabulary. Naturally, we choose the maximum as the prediction back to the model. The loss function in pre-training is defined as:

$$Loss_{pre} = -\frac{1}{|M|} \sum_{i=1}^{|M|} \sum_{j=1}^{|V|} y_{ij}^m \log(\phi_{ij}^m) \quad (6)$$

A Cross-entropy Loss is applied to measure the difference between the prediction and the truth. Equation 6 accumulates products of each predicting probability ϕ_{ij}^m logarithm and golden label y_{ij}^m (in vocabulary V for all masked tokens M), and the purpose of Cross Pre-training process is to reduce the loss.

3.5 Comments Generation

In Comments Generation, we collect outputs from Data Preparation, Review Lines Tagging, and Cross Pre-training, and then transfer them into review comments products.

1) Text Encoder: Now we successfully have code blocks v_i^s with review tags `review_tag` and a pre-trained model. Again, we use the tokenizer of T5 [43] to embed all text, including `review_tag` and the prefix. The T5 tokenizer encodes all text w_i into 768-dimensional vectors and adds an special end token `</s>`.

2) Transfer Learning: After fully pre-trained in Cross Pre-training, we further fine-tune the T5 cross encoder on text generation. Such generation is also known as transfer learning, where models trained on a dataset at a large scale are employed to solve a specific task [43, 68]:

$$\begin{aligned} f^{D_{\delta_k}}(x_i) &= \{P(y_i^D | x_i^D) | y_i^D \in Y^D, i = 1, 2, \dots, |X^D|\}, \\ f^{T_{\tau_k}}(x_i) &= \{P(y_i^T | x_i^T) | y_i^T \in Y^T, i = 1, 2, \dots, |X^T|\} \end{aligned} \quad (7)$$

In Equation 7, $f^{D_{\delta_k}}$ denotes the predicted conditional distributions the model output for the task δ_k in domain D , x_i^D is a domain observation in feature space X^D , while y_i^D is defined as a specific truth label in Y^D . The purpose of transfer learning is to utilize the knowledge in $f^{D_{\delta_k}}$ to improve $f^{T_{\tau_k}}$ effect on task τ_k in domain T . In this task, the ground truth here is the real comments from reviewers, which gets prepared in the first step.

Next, the loss function of comments generation is defined as:

⁸<https://huggingface.co/>

Table 1: Data Source

Repository	PRs(all)	PRs(suc)	Reviews(suc)
Graal	4,200	222	904
Dubbo	9,500	678	1,447
Netty	12,000	1,365	4,938
Apollo	4,200	98	305
Flink	18,200	4,759	23,274
Kafka	11,605	2,039	10,544
Skywalking	8,350	611	1,985
Redisson	4,100	49	82
Bazel	14,500	515	1,651
Jenkins	6,100	965	3,379
ElasticSearch	82,200	8,215	30,735
Total	174,955	19,516	79,344

$$Loss_{trans} = -\frac{1}{|S|} \sum_{i=1}^{|S|} \sum_{j=1}^{|V|} y_{ij}^s \log(\phi_{ij}^s) \quad (8)$$

Similar to the pre-training one, Equation 8 accumulates the difference between each predicting probability ϕ_{ij}^s and golden label y_{ij}^s for all generated sentences S . And for each sentence, the inference will finish at the prediction of the stop token or the max length.

3.6 Application

The Application is straightforward. We mainly leverage a fully trained model from Comments Generation as a synergy. When new code comes in modern code review, reviewers can freely select review lines first. After being tagged, this coded text is fed into the generation model and transferred into outputs. Finally, the generated comments are integrated into automatic review comments and returned to developers.

4 EXPERIMENTAL DESIGN

4.1 Research Questions

According to our research purpose, the evaluation aims at answering the following research questions:

- **RQ1:** How effectively does AUGER perform to generate review comments automatically, after synthetic training?
- **RQ2:** What is the contribution of sub-component designs to the overall performance in AUGER?
- **RQ3:** To what extent is the usefulness of comments AUGER generated, compared with manual ones?

We will specifically discuss research questions in Section 5 with the evaluation of AUGER.

4.2 Data Source

To make sure the quality of the dataset, we select 11 notable Java repositories from GitHub in top 60 stars: *Graal*⁹, *Dubbo*¹⁰, *Netty*¹¹

⁹<https://github.com/oracle/graal/>

¹⁰<https://github.com/apache/dubbo/>

¹¹<https://github.com/netty/netty/>

*Apollo*¹², *Flink*¹³, *Kafka*¹⁴, *Skywalking*¹⁵, *Redisson*¹⁶, *Bazel*¹⁷, *Jenkins*¹⁸, *ElasticSearch*¹⁹. What's more, these projects have at least 4,000 pull requests and 100 contributors. The detail of each repository is listed in Table 1. PRs(all) denotes the number of pull requests successfully got from repositories, PRs(suc) denotes the number of pull requests that contain code review comments, and Reviews(suc) denotes valid review nodes in Java.

4.3 Baselines

To better answer RQ1, we compare the performance of AUGER with some state-of-the-art baselines. Since we formulate the task as a text generation, we choose three methods that have been proven capable to handle text generation and a latest study of code review [9]:

- LSTM (Long Short-Term Memory) is a classic type of neural network to store information over extended time intervals [24]. Its novel gate units lessen error backflow sharply and achieves a significant performance on natural language generation [9].
- COPYNET addresses text generation into "sequence to sequence" learning with copying mechanism, which replicates input segments selectively to outputs [19].
- CodeBERT is a pre-training model proposed by Microsoft, which can effectively solve many downstream tasks in programming language [20].
- Recently, Tufano et al. made a study on code review automation and introduced a variant of T5 model to handle automatic code revision [57]. The model here we employed is their code-to-comment version.

4.4 Metrics

We use the ROUGE metric and Perfect Prediction rate to evaluate AUGER and its baselines. The ROUGE metric is widely used in Machine Translation evaluation [33]. We employ ROUGE-1 and ROUGE-L score, which measure the overlap of words and the longest sequence between hypothesis and reference. Specifically, ROUGE-1 has precision, recall, and F1-score scores on word level. Precision measures the ratio of the correct number in prediction to the amount, recall measures the ratio of the correct number in inference to all samples in truth, and F1-score is a harmonic mean of Precision and Recall. Finally, Perfect Prediction is the rate of forecasts completely the same as ideal ones.

4.5 Experiment Settings

This section introduces the main parameters we set in all experiments.

- 1) Data Partition. After data processing, we preserve those review comments from 3 to 128 length. Same as code text. Then we split

¹²<https://github.com/apollosconfig/apollo/>

¹³<https://github.com/apache/flink/>

¹⁴<https://github.com/apache/kafka/>

¹⁵<https://github.com/apache/skywalking/>

¹⁶<https://github.com/redisson/redisson/>

¹⁷<https://github.com/bazelbuild/bazel/>

¹⁸<https://github.com/jenkinsci/jenkins/>

¹⁹<https://github.com/elasticsearch/elasticsearch/>

Table 2: Training Hyperparameters

	Hyperparameter	Value
Pre-training Model	Learning rate	1e-4
	Max input length	256
	Batch size	16
Generation Model	Learning rate	1e-3
	Length penalty	1
	Max input length	128
	Max output length	128
	Batch size	32

our dataset into 80% training, 10% validation and 10% test, referring to Moreno-Torres et al. [39].

2) Pre-training Model. We employed a base T5 model fine-tuned on Java Code Documentation Generation Multitasks [17], and the model was trained in 60,000 steps and evaluated at every 6,000 step in pre-training. Some key hyperparameters are listed in Table 2.

3) Generation Model. Likewise, we selected the same T5 model to do the generation, and the model was trained in 28,000 steps and evaluated at every 2,000 step in fine-tuning. Table 2 presents some key hyperparameters we set.

4) Experiment Environment. We implemented all training with NVIDIA GeForce RTX 3090 GPU and CUDA 11.4²⁰, and it cost 12 hours and 5 hours to pre-train and fine-tunes AUGER, respectively.

5) Implementation of baselines. We leveraged LSTM and CopyNet built by AllenNLP²¹. The CodeBERT model comes from Hugging Face²² and the code-to-comment T5 variant comes from the repository of Tufano et al. [57].

5 EVALUATION

5.1 RQ1: Performance of generating comments

The results in Figure 4 show that AUGER outperforms baselines in every situation. We mainly focus on the Top-10 scores on every metric since they are theoretically the best for 1 to 10. In Top-10, AUGER achieves 22.97%, 4.32%, 26.11%, 27.07%, 25.28% on ROUGE-L, Perfect Prediction, Precision, Recall, and F1-score, and outperforms the best baseline by 37.38%, 14 times, 21.33%, 54.51%, and 38.22%, respectively. It indicates that our approach can generate review comments more precisely than baseline when fully trained, whether in a long statement or word similarity. In other cases shown in Figure 4, AUGER also achieves apparent outperformance from 11.95% up to tens of times as well.

We thoroughly analyze the results and explain the advantage of AUGER from two perspectives: 1) AUGER performs effectively on knowledge synthesis and text generation thanks to the advanced pre-training and transfer learning techniques. They provide a chance to learn knowledge from data at a large scale, whatever the language is. In that case, T5 is one of the most outstanding models. It outperforms traditional deep learning models like LSTM, CopyNET,

BERT on multi-tasks besides code-to-comment generation. 2) Especially, we address review comments generation into a fine-grained task compared with prior works, which inputs highlighted lines led by a review tag within code blocks, not functions themselves. Thus, Review Lines Tagging helps AUGER make pertinent review comments related to key code lines. The benefit will be proven in ablation (Section 5.2), too.

In addition, it is worth noting that the Perfect Prediction rate of all methods is low. There are two likely causes: 1) Perfect Prediction is indeed a strict metric that requires complete equality between inference and ground truth; 2) methods have limitations on solving this task. LSTM and CopyNET directly implement training on both programming and natural language, which, according to their output, mainly captures the semantics in comments. CodeBERT and Tufano et al. pre-train themselves on various data, including other programming or code-to-code language, which brings uncontrollable noise to their performance on this task. What's more, Tufano et al. trained their T5 without review tags, and it may mislead the model to irrelevant lines. Still, it struggles due to their defect-oriented dataset construction, too. LSTM and CopyNET perform worse for that k bigger than 3. It is mainly caused by the overfitting they suffer from when handling text in different languages.

For efficiency, AUGER automatically generates one piece of review comments in 20 seconds on average. Compared with manual ones made in minutes and hours, AUGER highly alleviates the burden of review process, i.e., inspecting, analyzing and commenting. What's more, AUGER works as a non-resting reviewer with immediate feedback, from which developers can receive review comments just in time. It effectively saves both developers and reviewers time of waiting due to the discrepancy of assignment.

We also present an example in Figure 5 for intuitive comparison. An instance of review code changes to input and review comments from all baselines are listed in Figure 5. The original code changes declare a protected function "close()" and the reviewer highlights a code line that monitors a statement "transformation.Chain.close()" and logs a warning message whenever the statement fails. Data Preparation and Review Lines Tagging will process the code into a code sentence as input. The second line shows that human reviewers find defects that the operation in the statement is "Transformation.close()" instead of "Transformation.stop" and make comments to point it out. Taking it as a reference, we compare each auto-output of machine techniques. AUGER performs the best and successfully find the defect by generating comments as same as a human. Comments from Tufano et al. and CodeBERT are in review language but confused. Although CopyNET and LSTM capture the defect on review lines, the outputs are ambiguous and useless opinions for submitters.

Answering RQ1: AUGER is capable to synthesize review knowledge and outperforms all baselines on five metrics when generating review comments automatically.

5.2 RQ2: Ablation experiment

In the ablation experiment, we first compare AUGER with two fundamental T5 versions widely used in research. These two models

²⁰<https://developer.nvidia.com/cuda-toolkit/>

²¹<https://allennlp.org/allennlp>

²²<https://huggingface.co/>

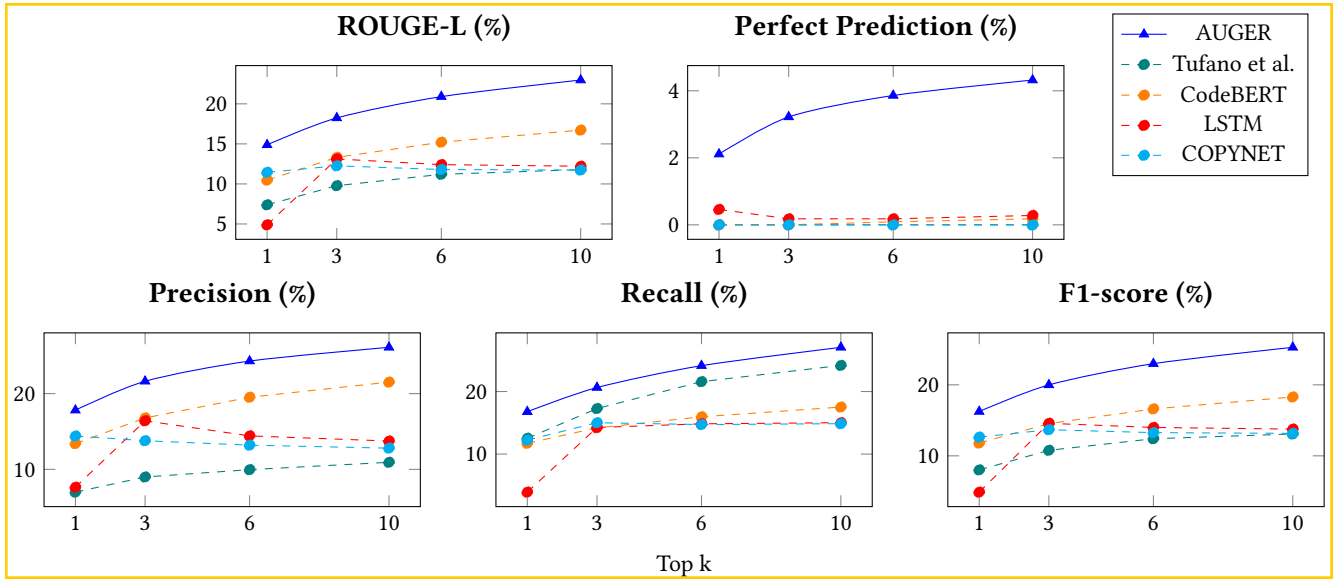


Figure 4: the Experiment Results on Top k Generation.

Review Code Changes	
Original:	<pre>protected void close() { synchronized (this) { tryStop(); } if (producer != null) { try { producer.close(30, TimeUnit.SECONDS); } catch (Throwable t) { log.warn("Could not stop producer", t); } } try { transformationChain.close(); } catch (Throwable t) { log.warn("Could not stop transformation chain", t); } }</pre>
Processed:	<pre>protected void close() { try { transformationChain.close(); } catch (Throwable t) { log.warn("Could not stop transformation chain", t); } }</pre>
Reviewers' Comments	
Nit:	the method is Transformation.close(), not "stop". The log message should probably reflect that.
AUGER	
	the method be transformation close not stop the log message should probably reflect that
Tufano et al.	
	If we're going to stop the review, maybe we should change this to info?
CodeBERT	
	stop method be producer close not stop the log message should probably reflect that
CopyNET	
	i think this should be stop a log chain
LSTM	
	i think this be a bug

Figure 5: an Example of AUGER

Table 3: the Ablation Results

Methods	ROUGE-L	Perfect Prediction
T5 base	22.01%	3.95%
T5 java	22.41%	4.14%
AUGER -<review_tag>	21.47%	3.31%
AUGER -pretraining	22.91%	3.95%
AUGER*	23.93%	4.04%
AUGER	22.97%	4.32%

results on the upper half of Table 3. These two models are the initial version that are fine-tuned with review tags and processes like word cut, rather than the one Tufano et al. trained with their raw data. The outperformance proved that AUGER could solve the problem more thoroughly than a simple fine-tuning on T5 models (-4.18% and -2.44%). We further remove Review Lines Tagging and Cross Pre-training parts, and the results are shown on the lower half (AUGER -<review_tag>, AUGER -pretraining). Similarly, both removals cause a decline in AUGER's performance (-6.53% and -0.26%).

To illustrate the unlimited synergy better, we warm boost AUGER at the half of training data and implement subsequent training for the rest. The result in the test (AUGER* in Table 3) shows nearly no efficiency loss and even an increase of ROUGE-L compared with the original one (AUGER). We believe the reason is that all data is randomized and more text related to the test occurs earlier in training steps. However, the overall performance remains stable and indicates that AUGER has the ability of training its synergy further.

are from Hugging Face²³. We trained the common T5-base and the Java pre-trained T5-base model on our dataset and listed their

²³<https://huggingface.co/>

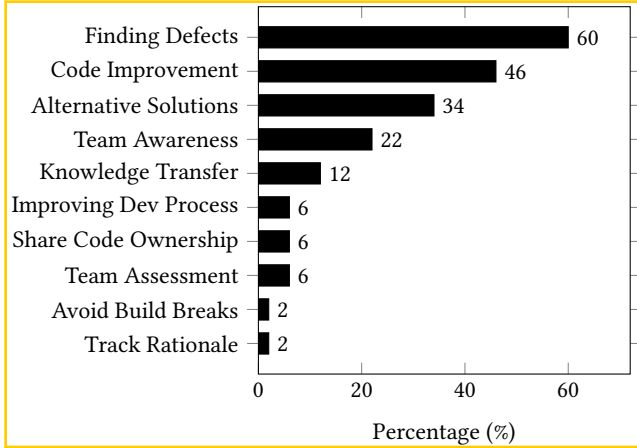


Figure 6: Motivation Reflection of Generated Comments

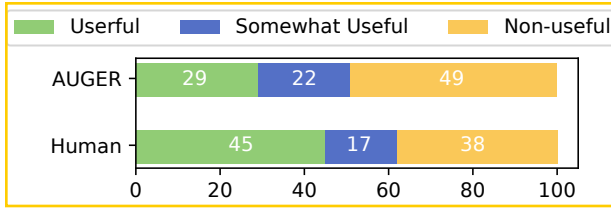


Figure 7: the Percentage of Useful Review Comments

The results illustrate that each component is contributive to the overall performance. The reasons are as follows: 1) As mentioned in Section 5.1, Review Lines Tagging narrows the scope down to highlighted review lines instead of a complete code file or function and saves models from deviated comprehension. 2) Cross Pre-training thoroughly trains the model with multiple steps and guides it to build the relationship between two different modalities, i.e., programming language and natural language. The improvement of these two modules is essential for AUGER. Notably, the warm boosting of AUGER (AUGER*) performs similarly, even better than AUGER. It indicates that AUGER is capable of working as an unlimited reviewer with subsequent training steps. As mentioned in Section 1, the scalability breaks the limitation of practical code review and sharply reduces the manual effort to grasp unfamiliar programming knowledge.

Answering RQ2: In the framework of AUGER, every component counts for the overall achievement and supports AUGER to train further.

5.3 RQ3: The Assessment of Usefulness

Subsections 5.1 and 5.2 demonstrate the ability of AUGER to handle the issues of code review practice being restricted and time-consuming. Furthermore, to ensure the effectiveness of AUGER, we sample 100 (1% of the best (not perfect) inference in the test and evaluate the usefulness. As mentioned in Section 1, AUGER

is our first step and aimed at commenting as effectively as human reviewers. Hence, according to Rahman et al. [44], we mainly compare it with manual review comments considered useful, which, in other words, indeed trigger source code changes on the author's side after reviewing (described in Section 3.2). The consideration follows the definition of Rahman et al [44]. In that case, the effectiveness has been evaluated early in the last two subsections (5.1 and 5.2) on automatic metrics that calculate the similarity between the generation and ground truth.

However, auto-metrics only calculate word similarity between products and ground truth, but no semantics. However, review comments should be understandable. We found that most products of traditional models, i.e., LSTM and CopyNet, are non-readable or suffer from repetition. They are far from use even though they are competitive in auto-metrics. AUGER aims to generate readable comments and make sense for code review, just like humans. Hence, though much difficult, we still compare the usefulness with manual ones, the only qualified reference.

We first employ the standard proposed by Baccheli et al. [2]. It reported that review comments are supposed to reflect ten of developers' motivations to drive code review. Some of them are for programming issues, while other motivations are for social benefits in several aspects. In that case, we analyze how effectively AUGER performs usefully to reflect them and present the result in Figure 6. The 100 samples successfully cover every aspect of valuable review comments as expected. The distribution is similar to Baccheli et al. of 570 actual data, too [2]: Finding Defects and Code Improvement are the most common reasons (60% and 46%) why developers drive code review, but other factors matter as well, such as Team Awareness of social benefits.

Next, we implement a classification of samples into three types: *useful*, *somewhat useful*, *non-useful*. It refers to the empirical study of Bosu et al. [8], in which they list several assessments for each type. Compared with manual ones, we also classify the ground truth of these 100 samples in the same criterion. For AUGER, it generates comments 29% useful, 22% somewhat useful and 49% non-useful. For reviewers, they perform better: 45% useful, 17% somewhat useful, and 38% non-useful. It seems reviewers present more accurate review comments when inspecting code changes themselves and outperform AUGER by 55.17% for useful comments. However, it decreases to 28.95% when calculating the overprediction of non-useful review comments. Especially, the criterion was built on the analysis of manual hand-writings and overweight the diversity of comment language. For example, they assumed that review comments were somewhat useful whenever they started with a "nitpick". But, AUGER characterizes the word as a special feature in review comments text and generate it frequently. What's more, Bosu et al. judged those of social benefits non-useful such as the impressive building of developers. Still, Baccheli et al. argued that it is of great importance for team collaboration [2].

5.4 Case Study

In this section, we conduct a case study to understand the effectiveness of AUGER to generate review comments better. We inspect the 100 samples in Section 5.3 and further analyze those not as perfect as humans when calculating Perfect Prediction. Among 96

"Wrong" Review Comments ? ① <pre>static FileSystemKind ... { ... <review_tag> if (scheme.startsWith("\s3") scheme.startsWith("\emr") scheme.startsWith("\oss")) ... }</pre> Reviewers: "Sounds good, I think hotfix will be better." AUGER: "i think this scheme should be a separate hot fix"
"Wrong" Review Comments ? ② <pre>public ... { <review_tag> void reset(); long compare(BaseRow inputRow, int inputIndex, BaseRow currentRow, int currentIndex); ... }</pre> Reviewers: "why a comparator need to be reset?" AUGER: "what do think about reset be"
"Wrong" Review Comments ? ③ <pre>public void addRecords (...) { <review_tag> log.info("{} Added records into the buffered queue of partition {}, new queue size is ...) ... }</pre> Reviewers: "Should this be debug level? Seems like this could be really spammy." AUGER: "can we just do this log to debug level"
"Wrong" Review Comments ? ④ <pre>private void deleteState(...) { <review_tag> ActionListener<Boolean> deleteModelStateListener = ActionListener.wrap(r -> ... }</pre> Reviewers: "Does UnusedStateRemover class need to be changed too?" AUGER: "could you revert this change since it be unrelated to the change"

Figure 8: Examples in Case Study

"wrong" review comments, we find 3 "nearly perfect" with only a different word. There are 4 "wrong" review comments delivering the same message as the manual one. We report them in Figure 8.

In Case No.1, reviewers judge the code change of "if" judgment practical and advise to raise a hotfix here instead of the merge request. Though not perfect, AUGER also reports a hotfix and successfully captures the critical variable "schema".

In Case No.2, reviewers propose a question about the meaning of the "reset" statement. Similarly, AUGER asks the author why to do "reset" here in another accent.

In Case No.3, reviewers point out that the "log" statement should be in a "debug level" since it's really "spammy". For AUGER, though not explaining the reason, it also reports the main issue and suggests setting the "log" to "debug level".

In Case No.4, reviewers think that the change of "UnusedStateRemover" is not necessary here. AUGER fails to point it out but still suggests removing the change on review lines similarly.

In conclusion, we totally scan the 100 review comments AUGER generates in the test and find 3 "nearly perfect" and 4 "perfect in meaning". In that case, we can estimate a latent improvement of AUGER's performance when fully analyzed.

Answering RQ3: Referring to two criteria of prior works, review comments generated by AUGER are nearly as useful as manual ones. It also reveals an underestimate for those "wrong" samples.

6 RELATED WORK

We focus our discussion on (i) studies on modern code review, (ii) pre-training models, and (iii) approaches to generating source code comments.

Modern Code Review. Several studies tried to automate modern code review process due to its time-consuming nature [32, 50]. Balachandran's method delivers comments automatically with the

integration of multiple static analysis tools [3]. Gupta et al. proposed DeepCodeReviewer (DCR), which uses deep learning to learn review knowledge and predict an ideal one from a repository [21]. Shi et al. tried to predict the approval of the revised code and proposed DACE with the performance of 48% F1-score [49]. Moreover, Tufano et al. recommended code changes directly using their automatic models with the 30% perfect prediction on their dataset [58].

AUGER, compared to the techniques discussed above, is able to generate review comments with given code changes and provide human-like support to modern code review.

Pre-training Models. Pre-training models have achieved remarkable success in natural language processing at present [6, 25, 41, 46]. Devlin et al. introduced a simple but powerful model named BERT (Bidirectional Encoder Representations from Transformers) to outperform others on eleven NLP tasks with large margin. The model even surpassed human's performance in challenging areas [15]. Following it, models trained in two stages have sprang up, i.e., learning representations at pre-training and fine-tuning on downstream tasks, and they are open to subsequent training. For example, RoBERTa [35], XLNET [66], and GPT [10]. In addition, models pre-trained on software texts become popular, with the potential to solve tasks like bug reporting [12, 48, 54].

Owing to the effectiveness of this pattern, recent studies work on exploring a framework for diverse unsupervised text data. It is also known as transfer learning in terms of implementation [27, 29, 42, 47]. Raffel et al. formulated all text-based language problem as a text-to-text problem and proposed a T5 model at large scale [43]. The model is designed based on the transformer architecture open to complicated input with self-attention layers, instead of RNNs or CNNs [11, 59]. By combining the knowledge from exploration with scale and new corpus, T5 characterized many tasks into text-to-text and achieved state-of-the-art results on benchmarks, including summarization, question answering and text classification [43].

Source Code Comments Generation. Studies on code comments and comprehension of programs can be traced back to 1980s

[53, 55, 64]. During software maintenance, good comments are of great importance to program understanding since developers are able to learn the meaning of code at ease with the help of this descriptive language [56]. However, developers sometimes do not comment their code adequately due to extra efforts, lack of relevant knowledge or overlooking the importance of code comments [51]. Study reported that developers have to spend up to 59% time on these manual activities, which limits the efficiency of software development heavily [65].

Automatic generation of source code comments just started in the last decade. The early methods focused on information retrieval, such as VSM algorithms [22, 23], clone detection [63], and LDA algorithms [40]. After that, with the prosperity of deep learning, Hu et al. [26] and Wan et al. [60] proposed their automatic models with deep neural networks. Recently, pre-training models including CodeBERT [18] and T5 [17] have been proved to outperform the state-of-the-art on comments generation.

Distinct from code comments that describe source code function, review comments are professional knowledge to reflect inappropriate code lines. Hence they have different focuses. Besides, all information code comments have to capture is just from its source code text. However, it requires numerous practices to make useful review comments. We implement similar code-to-comments generation but different in terms of programming comprehension.

7 THREATS TO VALIDITY

Construct validity: We use the original review comments written by reviewers as ground truth, assuming that they cause code revision. However, these comments have no guarantee to make sense for code improvement. For example, developers change their code by themselves right after code review activities. Therefore, datasets may contain some suboptimal review comments and affect the evaluation because we measure Perfect Prediction rate that only considers words entirely equal to manual ones. To partially address the threat, we manually analyzed a sample of non-perfect predictions in case study, calculating the percentage of valuable ones from different references.

Internal validity: A study shows that the impact of hyperparameters on T5 models remains unknown [43]. In that case, we fully explore prior works and follow them to set hyperparameters in our approach. We acknowledge that a few of other settings may cause better performance, which is also a part of our future study.

External validity: Although the dataset features thousands of instances, we limited our experiments to Java projects. Hence, we do not claim the generalizability for other programming languages. However, we select the notable systems with high-quality code reviews to train our model. Future work will further explore whether AUGER can learn knowledge of projects in different programming languages.

8 CONCLUSION

Since code review practice suffers from the limitation of individual collaboration, this paper proposed a model named AUGER, to generate review comments with pre-training T5 models automatically. We first fetch 79,344 Java reviews in Github and heuristically clean 85.29% noisy data considered useless or irrelevant, such as

regular replies. Then we build a framework leveraging Text-to-Text Transfer Transformers (T5) models to automatically integrate and generate review comments. The synergy captures the relationship between code and review language effectively and shows better performance than baselines and high efficiency to immediate feedback. The model is capable to train further and cover unfamiliar programs more freely than individuals. Several criteria from prior studies are also employed to assess the generation as useful human-like review comments to some extent.

We will further explore a language-agnostic implementation and a broad application to collaborate with professionals reviewing new systems in future work.

9 ACKNOWLEDGMENTS

We sincerely appreciate the valuable feedback from the anonymous reviewers. This work was supported by the Strategy Priority Research Program of Chinese Academy of Sciences (No.XDA20080-200), the National Key R&D Program of China (No.2021YFC3340204) and Chinese Academy of Sciences-Dongguan Science and Technology Service Network Plan (No. 202016002000032).

REFERENCES

- [1] Miltiadis Allamanis. 2019. The adverse effects of code duplication in machine learning models of code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153.
- [2] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)*. 719–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- [3] Vipin Balachandran. 2013. Reducing human effort and improving quality in peer code reviews using automatic static analysis and reviewer recommendation. In *2013 35th International Conference on Software Engineering (ICSE)*. 931–940. <https://doi.org/10.1109/ICSE.2013.6606642>
- [4] Gabriele Bavota and Barbara Russo. 2015. Four eyes are better than two: On the impact of code reviews on software quality. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 81–90. <https://doi.org/10.1109/ICSME.2015.7332454>
- [5] Kent Beck, Martin Fowler, and Grandma Beck. 1999. Bad smells in code. *Refactoring: Improving the design of existing code* 1, 1999 (1999), 75–88.
- [6] John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*. 120–128.
- [7] Amiangshu Bosu and Jeffrey C Carver. 2013. Impact of peer code review on peer impression formation: A survey. In *2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. IEEE, 133–142.
- [8] Amiangshu Bosu, Michaela Greiler, and Christian Bird. 2015. Characteristics of Useful Code Reviews: An Empirical Study at Microsoft. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. 146–156. <https://doi.org/10.1109/MSR.2015.21>
- [9] Raouf Boutaba, Mohammad A Salahuddin, Noura Limam, Sara Ayoubi, Nashid Shahriar, Felipe Estrada-Solano, and Oscar M Caicedo. 2018. A comprehensive survey on machine learning for networking: evolution, applications and research opportunities. *Journal of Internet Services and Applications* 9, 1 (2018), 1–99.
- [10] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [11] Jianpeng Cheng, Li Dong, and Mirella Lapata. 2016. Long short-term memory networks for machine reading. *arXiv preprint arXiv:1601.06733* (2016).
- [12] Agnieszka Ciborowska and Kostadin Damevski. 2021. Fast Changelog-based Bug Localization with BERT. *arXiv preprint arXiv:2112.14162* (2021).
- [13] Atacilio Cunha, Tayana Conte, and Bruno Gadelha. 2021. Code Review is Just Reviewing Code? A Qualitative Study with Practitioners in Industry. *Association for Computing Machinery*, New York, NY, USA, 269–274. <https://doi.org/10.1145/3474624.3477063>
- [14] Jacek Czerwotka, Michaela Greiler, and Jack Tilford. 2015. Code Reviews Do Not Find Bugs. How the Current Code Review Best Practice Slows Us Down. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. 27–28. <https://doi.org/10.1109/ICSE.2015.131>

- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [16] Ondřej Dušek, David M. Howcroft, and Verena Rieser. 2019. Semantic Noise Matters for Neural Natural Language Generation. In *Proceedings of the 12th International Conference on Natural Language Generation*. Association for Computational Linguistics, Tokyo, Japan, 421–426. <https://doi.org/10.18653/v1/W19-8652>
- [17] Ahmed Elnaggar, Wei Ding, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Silvia Severini, Florian Matthes, and Burkhard Rost. 2021. CodeTrans: Towards Cracking the Language of Silicon's Code Through Self-Supervised Deep Learning and High Performance Computing. *arXiv preprint arXiv:2104.02443* (2021).
- [18] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [19] Jiaran Gu, Zhengdong Lu, Hang Li, and Victor O.K. Li. 2016. Incorporating Copying Mechanism in Sequence-to-Sequence Learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Berlin, Germany, 1631–1640. <https://doi.org/10.18653/v1/P16-1154>
- [20] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).
- [21] Anshul Gupta and Neel Sundaresan. 2018. Intelligent code reviews using deep learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18) Deep Learning Day*.
- [22] Sonia Haiduc, Jairo Aponte, and Andrian Marcus. 2010. Supporting program comprehension with source code summarization. In *2010 ACM/IEEE 32nd International Conference on Software Engineering*, Vol. 2. 223–226. <https://doi.org/10.1145/1810295.1810335>
- [23] Sonia Haiduc, Jairo Aponte, Laura Moreno, and Andrian Marcus. 2010. On the Use of Automated Text Summarization Techniques for Summarizing Source Code. In *2010 17th Working Conference on Reverse Engineering*. 35–44. <https://doi.org/10.1109/WCRE.2010.13>
- [24] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [25] Jeremy Howard and Sebastian Ruder. 2018. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146* (2018).
- [26] Xing Hu, Ge Li, Xin Xia, David Lo, and Zhi Jin. 2018. Deep Code Comment Generation. In *2018 IEEE/ACM 26th International Conference on Program Comprehension (ICPC)*. 200–20010.
- [27] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyoukloong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019), 103–112.
- [28] Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. 2020. Spanbert: Improving pre-training by representing and predicting spans. *Transactions of the Association for Computational Linguistics* 8 (2020), 142–177.
- [29] Nitish Shirish Keskar, Bryan McCann, Lav R Varshney, Caiming Xiong, and Richard Socher. 2019. Ctrl: A conditional transformer language model for controllable generation. *arXiv preprint arXiv:1909.05858* (2019).
- [30] Huda Khayrallah and Philipp Koehn. 2018. On the Impact of Various Types of Noise on Neural Machine Translation. In *Proceedings of the 2nd Workshop on Neural Machine Translation and Generation*. Association for Computational Linguistics, Melbourne, Australia, 74–83. <https://doi.org/10.18653/v1/W18-2709>
- [31] Oleksii Kononenko, Olea Baysal, and Michael W. Godfrey. 2016. Code Review Quality: How Developers See It. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. 1028–1038. <https://doi.org/10.1145/2884781.2884840>
- [32] Oleksii Kononenko, Olea Baysal, and Michael W Godfrey. 2016. Code review quality: How developers see it. In *Proceedings of the 38th international conference on software engineering*. 1028–1038.
- [33] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. <https://aclanthology.org/W04-1013>
- [34] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2021. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *arXiv preprint arXiv:2107.13586* (2021).
- [35] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [36] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. 2022. Using Deep Learning to Generate Complete Log Statements. *arXiv preprint arXiv:2201.04837* (2022).
- [37] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The Impact of Code Review Coverage and Code Review Participation on Software Quality: A Case Study of the Qt, VTK, and ITK Projects. In *Proceedings of the 11th Working Conference on Mining Software Repositories (Hyderabad, India) (MSR 2014)*. Association for Computing Machinery, New York, NY, USA, 192–201. <https://doi.org/10.1145/2597073.2597076>
- [38] Rodrigo Morales, Shane McIntosh, and Foutse Khomh. 2015. Do code review practices impact design quality? A case study of the Qt, VTK, and ITK projects. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 171–180. <https://doi.org/10.1109/SANER.2015.7081827>
- [39] Jose Garcia Moreno-Llorces, Jose A. Saez, and Francisco Herrera. 2012. Study on the Impact of Partition-Induced Dataset Shift on 4-Fold Cross-Validation. *IEEE Transactions on Neural Networks and Learning Systems* 23, 8 (2012), 1304–1312. <https://doi.org/10.1109/TNNLS.2012.2199516>
- [40] Dana Movshovitz-Attias and William Cohen. 2013. Natural language models for predicting programming comments. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 35–40.
- [41] Alec Radford, Karthik Narasimhan, Jan LeSauter, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. (2018).
- [42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [43] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683* (2019).
- [44] Mohammad Masudur Rahman, Chanchal K. Roy, and Raula G. Kula. 2017. Predicting Usefulness of Code Review Comments Using Textual Features and Developer Experience. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. 215–226. <https://doi.org/10.1109/MSR.2017.17>
- [45] Peter C. Rigby and Christian Bird. 2013. Convergent Contemporary Software Peer Review Practices. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (Saint Petersburg, Russia) (ESEC/FSE 2013)*. Association for Computing Machinery, New York, NY, USA, 202–212. <https://doi.org/10.1145/2491411.2491444>
- [46] Justyna Sarzynska-Wawer, Aleksander Wawer, Aleksandra Pawlak, Julia Szymanowska, Izabela Stefaniak, Michal Jarkiewicz, and Lukasz Okruszek. 2021. Detecting formal thought disorder by deep contextualized word representations. *Psychiatry Research* 304 (2021), 114135.
- [47] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, Hyoungho Lee, Mingsheng Hong, Cliff Young, et al. 2018. Mesh-tensorflow: Deep learning for supercomputers. *arXiv preprint arXiv:1811.02084* (2018).
- [48] Lin Shi, Ziyu Jiang, Ye Yang, Xiao Chen, Yumin Zhang, Fangwen Mu, Hanzhi Jiang, and Qing Wang. 2021. ISPY: Automatic Issue-Solution Pair Extraction from Community Live Chats. *arXiv preprint arXiv:2109.07055* (2021).
- [49] Shu-Ting Shi, Ming Li, David Lo, Ferdian Thung, and Xuan Huo. 2019. Automatic code review by learning the revision of source code. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4910–4917.
- [50] Forrest Shull and Carolyn Seaman. 2008. Inspecting the history of inspections: An example of evidence-based technology diffusion. *IEEE software* 25, 1 (2008), 88–90.
- [51] Xiaotao Song, Hailong Sun, Xu Wang, and Jiafei Yan. 2019. A Survey of Automatic Generation of Source Code Comments: Algorithms and Techniques. *IEEE Access* 7 (2019), 111411–111428. <https://doi.org/10.1109/ACCESS.2019.2931579>
- [52] Davide Spadini, Gül Çalikli, and Alberto Bacchelli. 2020. Primers or Reminders? The Effects of Existing Review Comments on Code Review. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (Seoul, South Korea) (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1171–1182. <https://doi.org/10.1145/3377811.3380385>
- [53] Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori Pollock, and K. Vijay-Shanker. 2010. Towards Automatically Generating Summary Comments for Java Methods. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (Antwerp, Belgium) (ASE '10)*. Association for Computing Machinery, New York, NY, USA, 33–52. <https://doi.org/10.1145/1858996.1859006>
- [54] Jeniya Labassum, Mounica Maddela, Wei Xu, and Alan Ritter. 2020. Code and Named Entity Recognition in StackOverflow. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 4913–4926. <https://doi.org/10.18653/v1/2020.acl-main.443>
- [55] Ted Tenny. 1985. Procedures and Comments vs. the Banker's Algorithm. *SIGCSE Bull.* 17, 3 (sep 1985), 44–53. <https://doi.org/10.1145/382208.382523>
- [56] Ted Tenny. 1988. Program readability: Procedures versus comments. *IEEE Transactions on Software Engineering* 14, 9 (1988), 1271–1279.

- [57] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using Pre-Trained Models to Boost Code Review Automation. *arXiv preprint arXiv:2201.06850* (2022).
- [58] Rosalia Tufano, Luca Pascarella, Michele Tufanoy, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards Automating Code Review Activities. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 163–174.
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.
- [60] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S. Yu. 2018. *Improving Automatic Source Code Summarization via Deep Reinforcement Learning*. Association for Computing Machinery, New York, NY, USA, 397–407. <https://doi.org/10.1145/3238147.3238206>
- [61] Sinong Wang, Madian Khabza, and Hao Ma. 2020. To Pretrain or Not to Pretrain: Examining the Benefits of Pretraining on Resource Rich Tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 2209–2213. <https://doi.org/10.18653/v1/2020.acl-main.200>
- [62] Jason Wei and Kai Zou. 2019. Eda: Easy data augmentation techniques for boosting performance on text classification tasks. *arXiv preprint arXiv:1901.11196* (2019).
- [63] Edmund Wong, Taiyue Liu, and Lin Tan. 2015. CloCom: Mining existing source code for automatic comment generation. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 380–389. <https://doi.org/10.1109/SANER.2015.7081848>
- [64] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen. 1981. The Effect of Modularization and Comments on Program Comprehension. In *Proceedings of the 5th International Conference on Software Engineering* (San Diego, California, USA) (ICSE '81). IEEE Press, 215–223.
- [65] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E Hassan, and Shaping Li. 2017. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering* 44, 10 (2017), 951–976.
- [66] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. 2019. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems* 32 (2019).
- [67] Yue Yu, Huaimin Wang, Gang Yin, and Charles X. Ling. 2014. Reviewer Recommender of Pull-Requests in GitHub. In *2014 IEEE International Conference on Software Maintenance and Evolution*. 609–612. <https://doi.org/10.1109/ICSME.2014.107>
- [68] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. 2021. A Comprehensive Survey on Transfer Learning. *Proc. IEEE* 109, 1 (2021), 43–76. <https://doi.org/10.1109/JPROC.2020.3004555>